⚠️ Draft Standards Track: ERC

ERC-6900: Modular Smart Contract Accounts

Interfaces for smart contract accounts and modules, optionally supporting upgradability and introspection

Authors	Adam Egyed (@adamegyed), Fangting Liu (@trinity-0111), Jay Paik (@jaypaik), Yoav Weiss (@yoavw), Huawei Gu (@huaweigu), Daniel Lim (@dlim-circle), Ruben Koch (@0xrubes), David Philipson (@dphilipson), Howy Ho (@howydev), Nikita Belenkov (@nikita-quantstamp), zer0dot (@zer0dot), David Kim (@PowerStream3604)

Created	2023-04-18
Discussion Link	https://ethereum-magicians.org/t/eip-modular-smart-contract-accounts-and-plugins/13885
Requires	EIP-165, EIP-1271, EIP-4337

Table of Contents

- Abstract
- Motivation
- Specification
    - Terms
    - Overview
    - Interfaces
    - Validation Functions
    - Execution Functions
    - Hooks
    - Validation Call Flow
    - Execution Call Flow
    - Extension
- Rationale
    - ERC-4337 Dependency
    - Community Consensus
- Backwards Compatibility
- Reference Implementation
- Security Considerations
    - Wallet/SDK Developers
    - Module Developers

    - Users
- Copyright

Abstract

This proposal standardizes smart contract accounts and account modules, which are smart contracts that allow for composable logic within smart contract accounts. This proposal is compliant with ERC-4337. This standard emphasizes secure permissioning of modules, and maximal interoperability between all spec-compliant accounts and modules.

This modular approach splits account functionality into three categories, implements them in external contracts, and defines an expected execution flow from accounts.

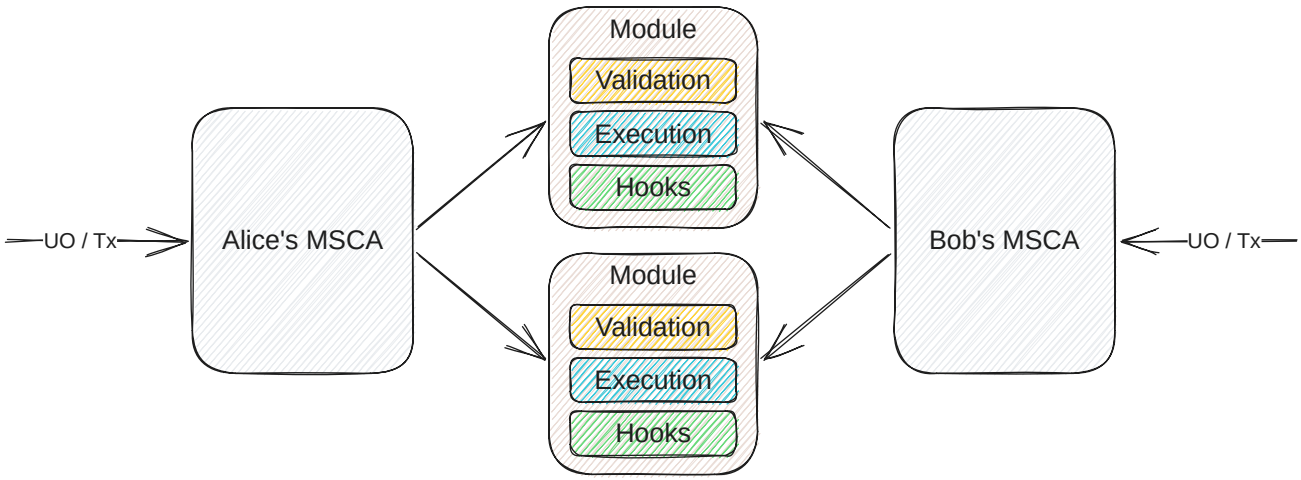
Motivation

One of the goals that ERC-4337 accomplishes is abstracting the logic for execution and validation to each smart contract account.

Many new features of accounts can be built by customizing the logic that goes into the validation and execution steps. Examples of such features include session keys, subscriptions, spending limits, and role-based access control. Currently, some of these features are implemented natively by specific smart contract accounts, and others are able to be implemented by proprietary module systems like Safe modules.

However, managing multiple account implementations provides a poor user experience, fragmenting accounts across supported features and security configurations. Additionally, it requires module developers to choose which platforms to support, causing either platform lock-in or duplicated development effort.

We propose a standard that coordinates the implementation work between module developers and account developers. This standard defines a modular smart contract account capable of supporting all standard-conformant modules. This allows users to have greater portability of their data, and for module developers to not have to choose specific account implementations to support.



These modules can contain execution logic, validation functions, and hooks. Validation functions define the circumstances under which the smart contract account will approve actions taken on its behalf, while hooks allow for pre and post execution controls.

Accounts adopting this standard will support modular, upgradable execution and validation logic. Defining this as a standard for smart contract accounts will make modules easier to develop securely and will allow for greater interoperability.

Specification

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “NOT RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in RFC 2119 and RFC 8174.

Terms

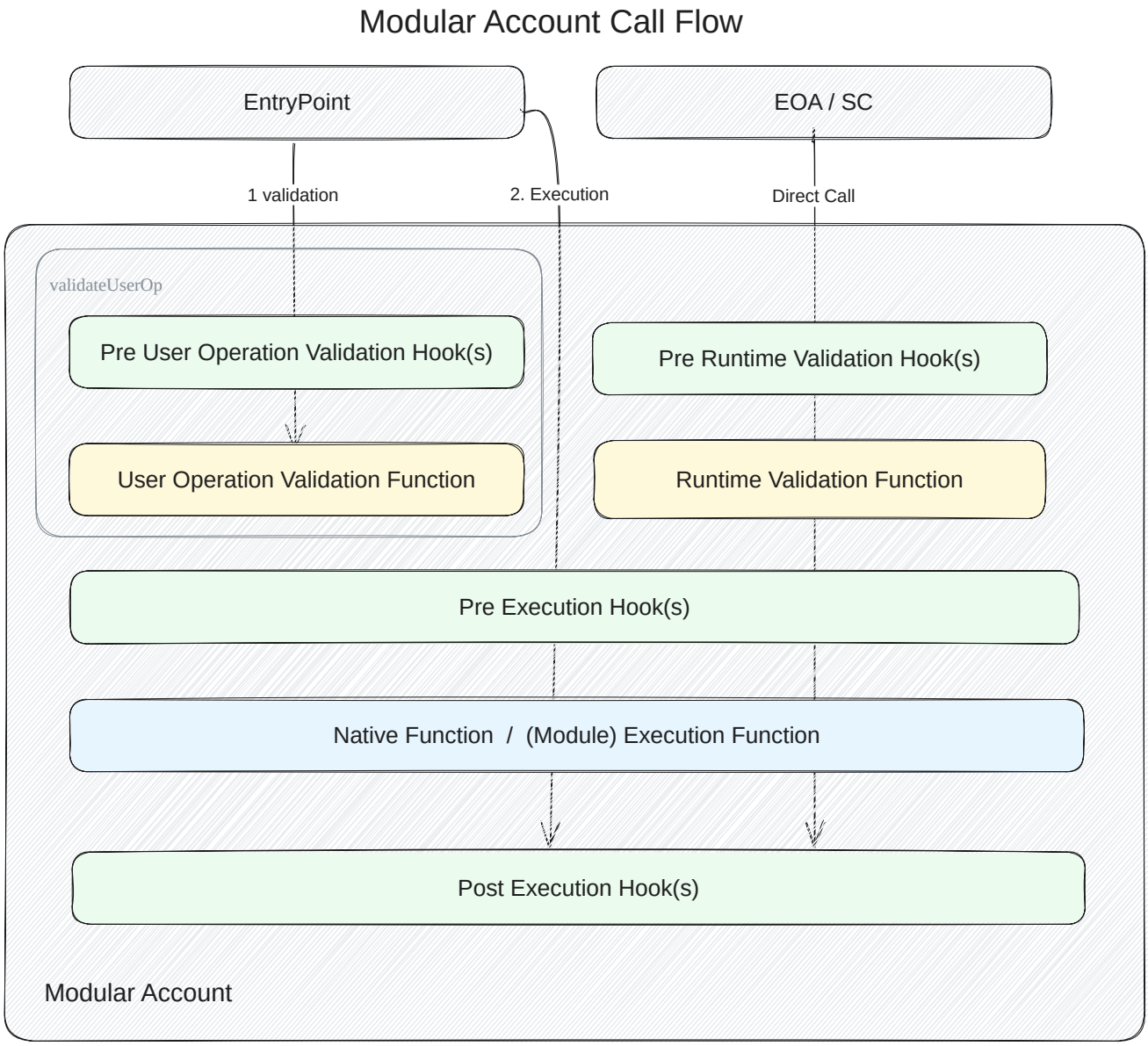
- An **account** (or **smart contract account, SCA**) is a smart contract that can be used to send transactions and hold digital assets. It implements the `IAccount` interface from ERC-4337.
- A **modular account** (or **modular smart contract account, MSCA**) is an account that supports modular functions. There are three types of modular functions:
    - **Validation functions** validate authorization on behalf of the account.
    - **Execution functions** execute custom logic allowed by the account.
    - **Hooks** execute custom logic and checks before and/or after an execution function or validation function. There are two types of hooks:
        - **Validation hooks** run before a validation function. These can enforce permissions on actions authorized by a validation function.
        - **Execution hooks** can run before and/or after an execution function. Execution hooks can be attached either to a specific execution function or a validation function. A pre execution hook may optionally return data to be consumed by a post execution hook.
- A **native function** refers to a function implemented by the modular account, as opposed to a function added by a module.
- A **module** is a deployed smart contract that hosts any amount of the above three kinds of modular functions.
- A module’s **manifest** describes the execution functions, interface IDs, and hooks that should be installed on the account.

Overview

A modular account handles two kinds of calls: either from the `EntryPoint` through ERC-4337, or through direct calls from externally owned accounts (EOAs) and other smart

contracts. This standard supports both use cases.

A call to the modular account can be broken down into the steps as shown in the diagram below. The validation steps validate if the caller is allowed to perform the call. The pre execution hook step can be used to do any pre execution checks or updates. It can also be used along with the post execution hook step to perform additional actions or verification. The execution step performs a defined task or collection of tasks.



Each step is modular, supporting different implementations, which allows for open-ended programmable accounts.

Interfaces

Modular accounts MUST implement:

- `IAccount.sol` and `IAccountExecute.sol` from ERC-4337.
- `IERC6900Account.sol` to support module management and usage, and account identification.
- The function `isValidSignature` from ERC-1271

Modular accounts MAY implement:

- `IERC6900AccountView.sol` to support visibility in account states on-chain.
- ERC-165 for interfaces installed from modules.

Modules MUST implement:

- `IERC6900Module.sol` described below and implement ERC-165 for `IERC6900Module`.

Modules MAY implement any of the following module types:

- `IERC6900ValidationModule` to support validation functions for the account.
- `IERC6900ValidationHookModule` to support hooks for validation functions.
- `IERC6900ExecutionModule` to support execution functions and their installations on the account.
- `IERC6900ExecutionHookModule` to support pre & post execution hooks for execution functions.

`IERC6900Account.sol`

Module execution and management interface. Modular accounts MUST implement this interface to support installing and uninstalling modules, and open-ended execution.

```
/// @dev A packed representation of a module function.
/// Consists of the following, left-aligned:
/// Module address: 20 bytes
/// Entity ID:      4 bytes
type ModuleEntity is bytes24;

/// @dev A packed representation of a validation function and its associated
/// Consists of the following, left-aligned:
/// Module address:     20 bytes
/// Entity ID:          4 bytes
/// ValidationFlags:    1 byte
type ValidationConfig is bytes25;

// ValidationFlags layout:
// 0b00000___ // unused
// 0b_____A__ // isGlobal
// 0b______B_ // isSignatureValidation
// 0b_______C // isUserOpValidation
type ValidationFlags is uint8;

/// @dev A packed representation of a hook function and its associated flags.
/// Consists of the following, left-aligned:
/// Module address: 20 bytes
/// Entity ID:      4 bytes
/// Flags:          1 byte
```

```
///
/// Hook flags layout:
/// 0b00000___ // unused
/// 0b_____A__ // hasPre (exec only)
/// 0b______B_ // hasPost (exec only)
/// 0b_______C // hook type (0 for exec, 1 for validation)
type HookConfig is bytes25;

struct Call {
    // The target address for the account to call.
    address target;
    // The value to send with the call.
    uint256 value;
    // The calldata for the call.
    bytes data;
}

interface IERC6900Account {
    event ExecutionInstalled(address indexed module, ExecutionManifest manife
    event ExecutionUninstalled(address indexed module, bool onUninstallSuccee
    event ValidationInstalled(address indexed module, uint32 indexed entityId
    event ValidationUninstalled(address indexed module, uint32 indexed entity

    /// @notice Standard execute method.
    /// @param target The target address for the account to call.
    /// @param value The value to send with the call.
    /// @param data The calldata for the call.
    /// @return The return data from the call.
    function execute(address target, uint256 value, bytes calldata data) exte

    /// @notice Standard executeBatch method.
    /// @dev If the target is a module, the call SHOULD revert. If any of the
    /// revert.
    /// @param calls The array of calls.
    /// @return An array containing the return data from the calls.
    function executeBatch(Call[] calldata calls) external payable returns (by

    /// @notice Execute a call using the specified runtime validation.
    /// @param data The calldata to send to the account.
    /// @param authorization The authorization data to use for the call. The
    /// specifies which runtime validation to use, and the rest is sent as a
    function executeWithRuntimeValidation(bytes calldata data, bytes calldata
        external
        payable
        returns (bytes memory);

    /// @notice Install a module to the modular account.
    /// @param module The module to install.
    /// @param manifest the manifest describing functions to install.
```

```
    /// @param installData Optional data to be used by the account to handle
    /// is implementation-specific.
    function installExecution(
        address module,
        ExecutionManifest calldata manifest,
        bytes calldata installData
    ) external;

    /// @notice Uninstall a module from the modular account.
    /// @param module The module to uninstall.
    /// @param manifest The manifest describing functions to uninstall.
    /// @param uninstallData Optional data to be used by the account to handl
    /// encoding is implementation-specific.
    function uninstallExecution(
        address module,
        ExecutionManifest calldata manifest,
        bytes calldata uninstallData
    ) external;

    /// @notice Installs a validation function across a set of execution sele
    /// global validation function.
    /// @param validationConfig The validation function to install, along wit
    /// @param selectors The selectors to install the validation function for
    /// @param installData Optional data to be used by the account to handle
    /// encoding is implementation-specific.
    /// @param hooks Optional hooks to install and associate with the validat
    /// implementation-specific.
    function installValidation(
        ValidationConfig validationConfig,
        bytes4[] calldata selectors,
        bytes calldata installData,
        bytes[] calldata hooks
    ) external;

    /// @notice Uninstall a validation function from a set of execution selec
    /// @param validationFunction The validation function to uninstall.
    /// @param uninstallData Optional data to be used by the account to handl
    /// encoding is implementation-specific.
    /// @param hookUninstallData Optional data to be used by the account to h
    /// is implementation-specific.
    function uninstallValidation(
        ModuleEntity validationFunction,
        bytes calldata uninstallData,
        bytes[] calldata hookUninstallData
    ) external;

    /// @notice Return a unique identifier for the account implementation.
    /// @dev This function MUST return a string in the format "vendor.account
    /// names MUST NOT contain a period character.
```

```
    /// @return The account ID.
    function accountId() external view returns (string memory);
}
```

`IERC6900AccountView.sol`

Module inspection interface. Modular accounts MAY implement this interface to support visibility in module configuration.

```
/// @dev Represents data associated with a specific function selector.
struct ExecutionDataView {
    // The module that implements this execution function.
    // If this is a native function, the address must be the address of the a
    address module;
    // Whether or not the function needs runtime validation, or can be called
    // state changing if this flag is set to true.
    // Note that even if this is set to true, user op validation will still b
    // drain the account of native tokens by wasting gas.
    bool skipRuntimeValidation;
    // Whether or not a global validation function may be used to validate th
    bool allowGlobalValidation;
    // The execution hooks for this function selector.
    HookConfig[] executionHooks;
}

struct ValidationDataView {
    // ValidationFlags layout:
    // 0b00000___ // unused
    // 0b_____A__ // isGlobal
    // 0b______B_ // isSignatureValidation
    // 0b_______C // isUserOpValidation
    ValidationFlags validationFlags;
    // The validation hooks for this validation function.
    HookConfig[] validationHooks;
    // Execution hooks to run with this validation function.
    HookConfig[] executionHooks;
    // The set of selectors that may be validated by this validation function
    bytes4[] selectors;
}

interface IERC6900AccountView {
    /// @notice Get the execution data for a selector.
    /// @dev If the selector is a native function, the module address will be
    /// @param selector The selector to get the data for.
    /// @return The execution data for this selector.
    function getExecutionData(bytes4 selector) external view returns (Executi

    /// @notice Get the validation data for a validation function.
```

```
    /// @dev If the selector is a native function, the module address will be
    /// @param validationFunction The validation function to get the data for
    /// @return The validation data for this validation function.
    function getValidationData(ModuleEntity validationFunction)
        external
        view
        returns (ValidationDataView memory);
}
```

`IERC6900Module.sol`

Module interface. Modules MUST implement this interface to support module management and interactions with ERC-6900 modular accounts.

```
interface IERC6900Module is IERC165 {
    /// @notice Initialize module data for the modular account.
    /// @dev Called by the modular account during `installExecution`.
    /// @param data Optional bytes array to be decoded and used by the module
    /// modular account.
    function onInstall(bytes calldata data) external;

    /// @notice Clear module data for the modular account.
    /// @dev Called by the modular account during `uninstallExecution`.
    /// @param data Optional bytes array to be decoded and used by the module
    /// account.
    function onUninstall(bytes calldata data) external;

    /// @notice Return a unique identifier for the module.
    /// @dev This function MUST return a string in the format "vendor.module.
    /// names MUST NOT contain a period character.
    /// @return The module ID.
    function moduleId() external view returns (string memory);
}
```

`IERC6900ValidationModule.sol`

Validation module interface. Modules MAY implement this interface to provide validation functions for the account.

```
interface IERC6900ValidationModule is IERC6900Module {
    /// @notice Run the user operation validation function specified by the `
    /// @param entityId An identifier that routes the call to different inter
    /// be more than one.
    /// @param userOp The user operation.
    /// @param userOpHash The user operation hash.
    /// @return Packed validation data for validAfter (6 bytes), validUntil (
    function validateUserOp(uint32 entityId, PackedUserOperation calldata use
```

```
        external
        returns (uint256);

    /// @notice Run the runtime validation function specified by the `entityI
    /// @dev To indicate the entire call should revert, the function MUST rev
    /// @param account The account to validate for.
    /// @param entityId An identifier that routes the call to different inter
    /// be more than one.
    /// @param sender The caller address.
    /// @param value The call value.
    /// @param data The calldata sent.
    /// @param authorization Additional data for the validation function to u
    function validateRuntime(
        address account,
        uint32 entityId,
        address sender,
        uint256 value,
        bytes calldata data,
        bytes calldata authorization
    ) external;

    /// @notice Validates a signature using ERC-1271.
    /// @dev To indicate the entire call should revert, the function MUST rev
    /// @param account The account to validate for.
    /// @param entityId An identifier that routes the call to different inter
    /// be more than one.
    /// @param sender The address that sent the ERC-1271 request to the smart
    /// @param hash The hash of the ERC-1271 request.
    /// @param signature The signature of the ERC-1271 request.
    /// @return The ERC-1271 `MAGIC_VALUE` if the signature is valid, or 0xFF
    function validateSignature(
        address account,
        uint32 entityId,
        address sender,
        bytes32 hash,
        bytes calldata signature
    ) external view returns (bytes4);
}
```

`IERC6900ValidationHookModule.sol`

Validation hook module interface. Modules MAY implement this interface to provide hooks for validation functions for the account.

```
interface IERC6900ValidationHookModule is IERC6900Module {
    /// @notice Run the pre user operation validation hook specified by the `
    /// @dev Pre user operation validation hooks MUST NOT return an authorize
    /// @param entityId An identifier that routes the call to different inter
```

```
    /// be more than one.
    /// @param userOp The user operation.
    /// @param userOpHash The user operation hash.
    /// @return Packed validation data for validAfter (6 bytes), validUntil (
    function preUserOpValidationHook(uint32 entityId, PackedUserOperation cal
        external
        returns (uint256);

    /// @notice Run the pre runtime validation hook specified by the `entityI
    /// @dev To indicate the entire call should revert, the function MUST rev
    /// @param entityId An identifier that routes the call to different inter
    /// be more than one.
    /// @param sender The caller address.
    /// @param value The call value.
    /// @param data The calldata sent.
    /// @param authorization Additional data for the hook to use.
    function preRuntimeValidationHook(
        uint32 entityId,
        address sender,
        uint256 value,
        bytes calldata data,
        bytes calldata authorization
    ) external;

    /// @notice Run the pre signature validation hook specified by the `entit
    /// @dev To indicate the call should revert, the function MUST revert.
    /// @param entityId An identifier that routes the call to different inter
    /// be more than one.
    /// @param sender The caller address.
    /// @param hash The hash of the message being signed.
    /// @param signature The signature of the message.
    function preSignatureValidationHook(uint32 entityId, address sender, byte
        external
        view;
}
```

IERC6900ExecutionModule.sol

Execution module interface. Modules MAY implement this interface to provide execution functions for the account.

```
struct ManifestExecutionFunction {
    // The selector to install.
    bytes4 executionSelector;
    // If true, the function won't need runtime validation, and can be called
    bool skipRuntimeValidation;
    // If true, the function can be validated by a global validation function
    bool allowGlobalValidation;
```

```
}

struct ManifestExecutionHook {
    bytes4 executionSelector;
    uint32 entityId;
    bool isPreHook;
    bool isPostHook;
}

/// @dev A struct describing how the module should be installed on a modular
struct ExecutionManifest {
    // Execution functions defined in this module to be installed on the MSCA
    ManifestExecutionFunction[] executionFunctions;
    ManifestExecutionHook[] executionHooks;
    // List of ERC-165 interface IDs to add to account to support introspecti
    // IERC6900Module's interface ID.
    bytes4[] interfaceIds;
}

interface IERC6900ExecutionModule is IERC6900Module {
    /// @notice Describe the contents and intended configuration of the modul
    /// @dev This manifest MUST stay constant over time.
    /// @return A manifest describing the contents and intended configuration
    function executionManifest() external pure returns (ExecutionManifest mem
}
```

IERC6900ExecutionHookModule.sol

Execution hook module interface. Modules MAY implement this interface to provide hooks for execution functions for the account.

```
interface IERC6900ExecutionHookModule is IERC6900Module {
    /// @notice Run the pre execution hook specified by the `entityId`.
    /// @dev To indicate the entire call should revert, the function MUST rev
    /// @param entityId An identifier that routes the call to different inter
    /// be more than one.
    /// @param sender The caller address.
    /// @param value The call value.
    /// @param data The calldata sent. For `executeUserOp` calls of validatio
    /// should receive the full calldata.
    /// @return Context to pass to a post execution hook, if present. An empt
    function preExecutionHook(uint32 entityId, address sender, uint256 value,
        external
        returns (bytes memory);

    /// @notice Run the post execution hook specified by the `entityId`.
    /// @dev To indicate the entire call should revert, the function MUST rev
    /// @param entityId An identifier that routes the call to different inter
```

```
    /// be more than one.
    /// @param preExecHookData The context returned by its associated pre exe
    function postExecutionHook(uint32 entityId, bytes calldata preExecHookDat
}
```

Validation Functions

Installation

- The account MUST install all validation hooks specified by the user and SHOULD call `onInstall` with the user-provided data on the hook module to initialize state if specified by the user.
- The account MUST install all execution hooks specified by the user and SHOULD call `onInstall` with the user-provided data on the hook module to initialize state if specified by the user.
- The account MUST configure the validation function to validate all of the selectors specified by the user.
- The account MUST set all flags as specified, like `isGlobal`, `isSignatureValidation`, and `isUserOpValidation`.
- The account SHOULD call `onInstall` on the validation module to initialize state if specified by the user.
- The account MUST emit `ValidationInstalled` as defined in the interface for all installed validation functions.

Uninstallation

During validation uninstallation, the account MUST correctly clear flags and other fields based on the incoming data provided by the user.

- The account MUST clear all flags for the validation function, like `isGlobal`, `isSignatureValidation`, and `isUserOpValidation`.
- The account MUST remove all hooks and SHOULD clear hook module states by calling `onUninstall` with the user-provided data for each hook, including both validation hooks and execution hooks, if specified by the user.
    - The account MAY ignore the revert from `onUninstall` with try/catch depending on the design principle of the account.
- The account MUST clear the configuration for the selectors that the validation function can validate.
- The account SHOULD call `onUninstall` on the validation module to clean up state if specified by the user.
- The account MUST emit `ValidationUninstalled` as defined in the interface for all uninstalled validation functions.

Execution Functions

Installation

- The account MUST install all execution functions and set flags and fields as specified in the manifest.
    - An execution function selector MUST be unique in the account.
    - An execution function selector MUST not conflict with native ERC-4337 and ERC-6900 functions.
- The account MUST add all execution hooks as specified in the manifest.
- The account SHOULD add all supported interfaces as specified in the manifest.
- The account SHOULD call `onInstall` on the execution module to initialize state if specified by the user.
- The account MUST emit `ExecutionInstalled` as defined in the interface for all installed executions.

Uninstallation

During execution uninstallation, the account MUST correctly clear flags and other fields based on the incoming data and module manifest provided by the user.

- The account MUST remove all execution functions and clear flags and fields as specified in the manifest.
- The account MUST remove all execution hooks as specified in the manifest.
- The account SHOULD remove all supported interfaces as specified in the manifest.
- The account SHOULD call `onUninstall` on the execution module to clean up state and track call success if specified by the user.
- The account MUST emit `ExecutionUninstalled` as defined in the interface for all uninstalled executions.

Hooks

Execution Hooks Data Format

For accounts that implement execution hooks, accounts MUST conform to these execution hook formats:

1. For `executeUserOp` calls, for execution hooks associated with a validation function, accounts MUST send the full calldata (`msg.data` in solidity), including the `executeUserOp` selector.
2. For `executeUserOp` calls, for execution hooks associated with a selector, accounts MUST send `PackedUserOperation.callData` for `executeUserOp` calls, excluding `executeUserOp.selector` and the rest of the `PackedUserOperation`.
3. For `executeWithRuntimeValidation` calls, for all execution hooks, accounts MUST send the inner `data` field.
4. For all other calls, for execution hooks associated with a selector, accounts MUST send over the full calldata (`msg.data` in solidity).

Hook Execution Order

It is RECOMMENDED that an account implementer runs hooks in first installed first executed order. However, an account MAY implement a different execution order.

Validation Call Flow

Modular accounts support three different calls flows for validation: user op validation, runtime validation, and signature validation. User op validation happens within the account's implementation of the function `validateUserOp`, defined in the ERC-4337 interface `IAccount`. Runtime validation happens through the dispatcher function `executeWithRuntimeValidation`, or when using direct call validation. Signature validation happens within the account's implementation of the function `isValidSignature`, defined in ERC-1271.

For each of these validation types, an account implementation MAY specify its own format for selecting which validation function to use, as well as any per-hook data for validation hooks.

Within the implementation of each type of validation function, the modular account MUST check that the provided validation function applies to the given function selector intended to be run (See Checking Validation Applicability). Then, the account MUST execute all validation hooks of the corresponding type associated with the validation function in use. After the execution of validation hooks, the account MUST invoke the validation function of the corresponding type. If any of the validation hooks or the validation function reverts, the account MUST revert. It SHOULD include the module's revert data within its revert data.

The account MUST define a way to pass data separately for each validation hook and the validation function itself. This data SHOULD be sent as the `userOp.signature` field for user op validation, the `authorization` field for runtime validation, and the `signature` field for signature validation.

The result of user op validation MUST be the intersection of time bounds returned by the validation hooks and the validation function. If any validation hooks or the validation functions returns a value of `1` for the authorizer field, indicating a signature verification failure by the ERC-4337 standard, the account MUST return a value of `1` for the authorizer portion of the validation data.

The set of validation hooks run MUST be the hooks specified by account state at the start of validation. In other words, if the set of applicable hooks changes during validation, the original set of hooks MUST still run, and only future invocations of the same validation should reflect the changed set of hooks.

Checking Validation Applicability

To enforce module permission isolation, the modular account MUST check validation function applicability as part of each validation function implementation.

User op validation and runtime validation functions have a configurable range of applicability to functions on the account. This can be configured with selectors installed to a validation. Alternatively, a validation installation MAY specify the `isGlobal` flag as true, which means the account MUST consider it applicable to any module execution function with the `allowGlobalValidation` flag set to true, or for any account native function that the account MAY allow for global validation.

If the selector being checked is `execute` or `executeBatch`, the modular account MUST perform additional checking. If the target of `execute` is the modular account's own address, or if the target of any `Call` within `executeBatch` is the account, validation MUST either revert or check that validation applies to the selector(s) being called.

Installed validation functions have two additional flag variables indicating what they may be used for. If a validation function is attempted to be used for user op validation and the flag `isUserOpValidation` is set to false, validation MUST revert. If the validation function is attempted to be used for signature validation and the flag `isSignatureValidation` is set to false, validation MUST revert.

Direct Call Validation

If a validation function is installed with the entity ID of `0xffffffff`, it may be used as direct call validation. This occurs when a module or other address calls a function on the modular account, without wrapping its call in the dispatcher function `executeWithRuntimeValidation` to use as a selection mechanism for a runtime validation function.

To implement direct call validation, the modular account MUST treat direct function calls that are not from the modular account itself or the `EntryPoint` as an attempt to validate using the caller's address and the entity ID of `0xffffffff`. If such a validation function is installed, and applies to the function intended to be called, the modular account MUST allow it to continue, without performing runtime validation. Any validation hooks and execution hooks installed to this validation function MUST still run.

Execution Call Flow

For all non-view functions within `IERC6900Account` except `executeWithRuntimeValidation`, all module-defined execution functions, and any additional native functions that the modular account MAY wish to include, the modular account MUST adhere to these steps during execution:

If the caller is not the `EntryPoint` or the account, the account MUST check access control for direct call validation.

Prior to running the target function, the modular account MUST run all pre execution hooks that apply for the current function call. Pre execution hooks apply if they have been installed to the currently running function selector, or if they are installed as an execution

hook to the validation function that was used for the current execution. Pre execution hooks MUST run validation-associated hooks first, then selector-associated hooks second.

Next, the modular account MUST run the target function, either an account native function or a module-defined execution function.

After the execution of the target function, the modular account MUST run any post execution hooks. These MUST be run in the reverse order of the pre execution hooks. If a hook is defined to be both a pre and a post execution hook, and the pre execution hook returned a non-empty `bytes` value to the account, the account MUST pass that data to the post execution hook.

The set of hooks run for a given target function MUST be the hooks specified by account state at the start of the execution phase. In other words, if the set of applicable hooks changes during execution, the original set of hooks MUST still run, and only future invocations of the same target function should reflect the changed set of hooks.

Module execution functions where the field `skipRuntimeValidation` is set to true, as well as native functions without access control, SHOULD omit the runtime validation step, including any runtime validation hooks. Native functions without access control MAY also omit running execution hooks.

Extension

Semi-Modular Account

Account implementers MAY choose to design a semi-modular account, where certain features, such as default validation, are integrated into the core account. This approach SHOULD ensure compatibility with fully modular accounts, as defined in this proposal, to maintain interoperability across different implementations.

Rationale

ERC-4337 compatible accounts must implement the `IAccount` interface, which consists of only one method that bundles validation with execution: `validateUserOp`. A primary design rationale for this proposal is to extend the possible functions for a smart contract account beyond this single method by unbundling these and other functions, while retaining the benefits of account abstraction.

This proposal includes several interfaces that build on ERC-4337. First, we standardize a set of modular functions that allow smart contract developers greater flexibility in bundling validation, execution, and hook logic. We also propose interfaces that provide methods for querying execution functions, validation functions, and hooks on a modular account. The rest of the interfaces describe a module's methods for exposing its modular functions and desired configuration, and the modular account's methods for installing and removing modules and allowing execution across modules and external addresses.

ERC-4337 Dependency

ERC-6900's main objective is to create a secure and interoperable foundation through modular accounts and modules to increase the velocity and security of the smart account ecosystem, and ultimately the wallet ecosystem. Currently, the standard prescribes ERC-4337 for one of its modular account call flows. However, this does not dictate that ERC-6900 will continue to be tied to ERC-4337. It is likely that smart account builders will want to develop modular accounts that do not use ERC-4337 in the future (e.g., native account abstraction on rollups). Moreover, it is expected that ERC-4337 and its interfaces and contracts will continue to evolve until there is a protocol-level account abstraction.

In the current state of the AA ecosystem, it is tough to predict the direction the builders and industry will take, so ERC-6900 will evolve together with the space's research, development, and adoption. The standard will do its best to address the objectives and create a secure foundation for modular accounts that may eventually be abstracted away from the infrastructure mechanism used.

Community Consensus

While this standard has largely been the result of collaboration among the coauthors, there have been noteworthy contributions from others in the community with respect to improvements, education, and experimentation. Thank you to the contributors:

- Gerard Persoon (@gpersoon)
- Harry Jeon (@sm-stack)
- Zhiyu Zhang (@ZhiyuCircle)
- Danilo Neves Cruz (@cruzdanilo)
- Iván Alberquilla (@ialberquilla)

We host community calls and working groups to discuss standard improvements and invite anyone with questions or contributions into our discussion.

Backwards Compatibility

Existing accounts that are deployed as proxies may have the ability to upgrade account implementations to one that supports this standard for modularity. Depending on implementation logic, existing modules may be wrapped in an adapter contract to adhere to the standard.

The standard also allows for flexibility in account implementations, including accounts that have certain features implemented without modules, so usage of modules may be gradually introduced.

Reference Implementation

See `https://github.com/erc6900/reference-implementation`

Security Considerations

Wallet/SDK Developers

From the wallet side, there are certain checks that the wallet/SDK should perform on the UI/UX side.

The standard introduces a concept of the execution manifest, which describes the execution functions, interface IDs, and hooks that should be installed on the account from an execution manifest. As part of constructing parameters to `installExecution()`, the SDK creates an `executionManifest` parameter that will specify the actions of the execution module being installed. SDKs will process the provided `executionManifest()` function of the underlying module, but the finished parameter will not necessarily be identical to the module's return value of `executionManifest()`. Furthermore, this parameter is only going through very limited checks on-chain as part of the installation process. Therefore, the `executionManifest` parameter needs to be carefully constructed and verified before being used for installations. Furthermore, the `executionManifest` parameter of uninstallation should ideally be the same used in installation to not leave residual data.

The standard supports a `isGlobalValidation` flag for validation functions, which means that this function is added to a global validation pool and can validate any of the execution functions that expose themselves to global validation via `allowGlobalValidation` flag. Depending on the implementation, some, all or none of the native execution functions could be globally validated. Therefore, the wallets should be careful about what validation functions they allow to be installed with global validation enabled, as that would allow these functions to validate the exposed native functions and hence bypass any restrictions that may have been added to protect these native/execution functions. In a sense, these global validation functions could gain root access to the exposed native execution functions and potentially the whole account.

Module Developers

The standard does not enforce any rules surrounding what data needs to be installed or uninstalled when a module is added/deleted from the account. Hence the `onUninstall()` function in various modules may leave behind residual state data, especially since the external call may not be performed all. Furthermore, execution hooks linked to an uninstalled function may remain configured. This could pose security risks or lead to unexpected behavior in the case where the module is reinstalled with the same `entityId`. The danger is that previously set permissions or data may be unintentionally reused. Hence it is a good idea to fully uninstall all data linked to the smart module account including execution hooks.

Users

The modular smart contract accounts themselves are trusted components. Installed modules are trusted to varying degrees, as modules can interact with an arbitrarily large or small set of resources on an account. For example, a wide-reaching malicious module could add reverting hooks to native function selectors, bricking the account, or add execution functions that may drain the funds of the account. However, it is also possible to install a module with a very narrow domain, and depend on the correctness of the account behavior to enforce its limited access. Users should, therefore be careful in what modules to add to their account.

Copyright

Copyright and related rights waived via CC0.

Citation

Please cite this document as:

Adam Egyed (@adamegyed), Fangting Liu (@trinity-0111), Jay Paik (@jaypaik), Yoav Weiss (@yoavw), Huawei Gu (@huaweigu), Daniel Lim (@dlim-circle), Ruben Koch (@0xrubes), David Philipson (@dphilipson), Howy Ho (@howydev), Nikita Belenkov (@nikita-quantstamp), zer0dot (@zer0dot), David Kim (@PowerStream3604), "ERC-6900: Modular Smart Contract Accounts [DRAFT]," *Ethereum Improvement Proposals*, no. 6900, April 2023. Available: https://eips.ethereum.org/EIPS/eip-6900.